

Contents

Decisiones de diseño y backing teórico	1
1. Función de activación	2
Lo que dice la clase	2
Lo que hicimos	2
Mismatch / sin backing directo	2
2. Inicialización de pesos	2
Lo que dice la clase	2
Lo que hicimos	2
Mismatch / sin backing directo	3
3. Optimizador	3
Lo que dice la clase	3
Lo que hicimos	3
Mismatch / sin backing directo	4
4. Learning rate	4
Lo que dice la clase	4
Lo que hicimos	4
Sin mismatch — alineado con el espíritu del PDF	4
5. Estrategia online / batch / mini-batch	4
Lo que dice la clase	4
Lo que hicimos	4
Sin mismatch — pero decisión “implícita”	5
6. Arquitectura: capas (M) y neuronas	5
Lo que dice la clase	5
Lo que hicimos	5
Mismatch / sin backing directo	6
7. Epochs y criterio de convergencia (epsilon)	6
Lo que dice la clase	6
Lo que hicimos	6
Sin mismatch — alineado con el espíritu del PDF	6
8. Bias	6
Lo que dice la clase	6
Lo que hicimos	6
Sin mismatch — pero “no penalizar bias en L2” es decisión sin backing PDF	7
9. Regularización / “Pack C” — matching con la clase	7
Mapeo 1:1 — lo que la clase enseña vs. lo que implementamos	7
Lo que la clase deja afuera (pero usamos)	7
Resultado del experimento (resumido del Ej3 README)	7
Pregunta de ppt_notes.md — “¿qué decisiones se tomaron?”	8
Resumen ultra-corto (cheatsheet mental)	8

Decisiones de diseño y backing teórico

Documento de referencia interno: para cada decisión que tomamos, qué dice la cátedra (lo que sí está en los materiales: PDF de regularización, PDF de optimizadores, HTML de weight init que vino con el TP) y qué hicimos nosotros. Cuando hay mismatch o decisión sin respaldo directo, está marcado.

Materiales de la cátedra a los que accedo: - docs/clase_regularizacion/regularizacion.pdf (27 slides) + 2 VTTs - docs/clase_optimizadores/clase_optimizadores.pdf (13 slides) + VTT - docs/weight_initialization/...html (artículo LinkedIn que la cátedra mandó como referencia)

Materiales que NO tengo: apunte de Narias sobre MLP/backprop, slides de clases anteriores. Cuando una decisión depende de “lo que dice el apunte” sin que yo lo pueda verificar, lo aclaro.

1. Función de activación

Lo que dice la clase

No hay un slide/PDF dedicado a activaciones en los materiales que tengo. El slide 2 de `regularizacion.pdf` (“Repaso”) menciona perceptrón simple **escalón / lineal / no lineal** y perceptrón multicapa, lo cual implica que el apunte previo (que no tengo) discutió las activaciones clásicas: escalón, lineal, sigmoide, tanh.

Lo que hicimos

Dónde	Activación	Justificación operativa
Ej0 — AND (perceptrón simple)	escalón (signo)	Salida discreta {-1, +1}; no requiere derivable porque no hay backprop.
Ej0 — XOR (MLP)	tanh	Bipolar {-1, +1} y derivable para backprop.
Ej1 — distillation	sigmoide	Target en [0, 1] (probabilidad de fraude). La sigmoide naturalmente devuelve ese rango.
Ej2/3 — capas ocultas	ReLU	Gradiente fuerte y constante en zona positiva; no satura como tanh/sigmoide → entrena más rápido.
Ej2/3 — capa de salida	softmax	Clasificación multiclase (10 dígitos): devuelve distribución de probabilidad.

Mismatch / sin backing directo

ReLU no aparece en los PDFs que tengo. El uso de ReLU en capas ocultas es la elección estándar moderna de la literatura (Glorot & Bengio 2011, “Deep Sparse Rectifier Neural Networks”) pero no la pude rastrear a un slide concreto de la cátedra. Es plausible que el apunte de Narias la mencione — no lo puedo verificar.

Softmax tampoco está en los PDFs. Para clasificación multiclase es la elección estándar; en el código `mlp/network.py:39–44` hay un guard que obliga a `loss=cross_entropy` cuando `activations[-1]=softmax`, porque la combinación tiene un gradiente especialmente limpio ($\partial L / \partial z = 0 - y$).

2. Inicialización de pesos

Lo que dice la clase

Fuente principal: `docs/weight_initialization/...html` (artículo LinkedIn collaborative que la cátedra mandó como referencia). Cubre explícitamente:

- **Random init** rompe la simetría entre neuronas, pero si la escala no es la adecuada produce **vanishing / exploding gradients**.
- **Xavier (Glorot):** `scale = sqrt(2 / (n_in + n_out))`. Mantiene varianza estable de activaciones y gradientes. Recomendado para **sigmoide/tanh**.
- **He (Kaiming):** `scale = sqrt(2 / n_in)`. Recomendado para **ReLU**: dobla la varianza de Xavier para compensar que ReLU “mata” la mitad del input. Evita “dying ReLU”.

Los PDFs de regularización y optimizadores **no mencionan inicialización**.

Lo que hicimos

Tres inicializadores en `mlp/initializers.py`:

Esquema	Fórmula en código	Cuándo lo usamos
Uniform	$U[-0.1, +0.1]$	Ej1 (perceptrón simple) y modo auto para activación identity.
He	$N(0, \sqrt{2/\text{fan_in}})$	Capas ReLU (modo auto). Coincide con HTML.
Xavier	$N(0, \sqrt{1/\text{fan_in}})$	Capas tanh/sigmoide/softmax (modo auto).

Modo auto en `mlp/initializers.py:25-33` mapea activación $\rightarrow$ init: - relu $\rightarrow$ he - tanh, sigmoid, softmax $\rightarrow$ xavier - identity $\rightarrow$ uniform

Esto contesta la pregunta de ppt_notes.md (“¿de dónde sacaste los initializers?”): de He et al. 2015 ($\sqrt{2/\text{fan_in}}$) y la familia Xavier/Glorot 2010, ambos cubiertos explícitamente en el HTML que vino con el TP.

Mismatch / sin backing directo

Nuestro Xavier usa $\sqrt{1/\text{fan_in}}$, no la fórmula del HTML $\sqrt{2/(\text{n_in} + \text{n_out})}$. Lo nuestro está más cerca de la “LeCun normal init” o de la versión de Xavier que solo usa `fan_in`. Ambas formas existen en la literatura y en frameworks (PyTorch tiene las dos), pero **si alguien compara nuestra fórmula con el HTML, no coinciden literalmente**. Empíricamente no hizo diferencia: en el sweep Fase 2 (Ej2 README) los tres init quedaron en empate técnico (~96%, std ~0.3%).

Uniform $[-0.1, 0.1]$ es ad-hoc. No tiene backing teórico, es la baseline simple del Ej1. La regla del HTML solo dice “valores pequeños, no cero” — eso lo cumple.

3. Optimizador

Lo que dice la clase

Fuente: `docs/clase_optimizadores/clase_optimizadores.pdf` (13 slides). Contenido exacto:

1. **Gradient descent** clásico: $\Delta w = -\eta \partial E / \partial w$. El problema es que la información es local.
2. **Momentum:** $\Delta w_{ij}(t+1) = -\eta \partial E / \partial w_{ij} + \alpha \Delta w_{ij}(t)$, con **α típico = 0.8 o 0.9**. Acumula velocidad en regiones planas, compensa oscilaciones en valles.
3. **η adaptativo:** ajustar η durante entrenamiento. Si el error decrece consistentemente, **subir η** ; si empieza a aumentar, **bajar η** ($\Delta \eta = +a$ o $-b\eta$).
4. **RMSProp:** $S_t = \gamma S_{t-1} + (1-\gamma) g_t^2$, $\Delta w = -\eta / \sqrt{S_t + \epsilon} \cdot g_t$. Ajusta LR por la RMS del gradiente.
5. **Adam** (Kingma & Ba 2015): combinación de Momentum + RMSProp. **“Muy usado en la práctica”**. Defaults del paper: **$\alpha = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$** .

Lo que hicimos

Los tres en `mlp/optimizers.py`: - **SGD** vanilla. - **Momentum** con $\beta=0.9$ (default del slide). - **Adam** con $\alpha=0.001$, $\beta_1=0.9$, $\beta_2=0.999$, $\epsilon=10^{-8}$ (defaults del paper, idénticos al slide).

Comparación empírica (Ej2 Fase 1.2):

Optimizador	val_acc	best_epoch	Comentario
Adam	96.74%	7	Ganador.
Momentum ($\beta=0.9$)	96.34%	18	Cerca, converge más lento.
SGD	94.81%	48 (sigue mejorando)	No convergió en 50 epochs.

Elegimos Adam. Backing: el slide del PDF dice textual “muy usado en la práctica”; nuestros números lo confirman.

Mismatch / sin backing directo

No implementamos η adaptativo (la regla $\Delta\eta = +a$ o $-b\eta$ que muestra el PDF). En su lugar implementamos `lr_schedule` tipo **step decay** (`lr *= decay` cada `N epochs`) en `mlp/network.py:191–197`. Está en la familia de “modificar η durante el entrenamiento” pero no es la regla específica del slide. Decisión razonable: step decay es la más común en la práctica moderna.

No implementamos RMSProp puro. Lo “absorbe” Adam (que combina RMSProp + Momentum). El PDF lo presenta como motivación de Adam; saltarse RMSProp no es contra el PDF.

4. Learning rate

Lo que dice la clase

No hay un slide específico de “elección de LR” en los PDFs. Lo que sí hay:

- En el PDF de optimizadores, el slide de **η adaptativo** dice que un η “demasiado grande” causa divergencia y uno “demasiado chico” no avanza — la motivación del η adaptativo.
- La recomendación de “barrer varios órdenes de magnitud” para LR es genérica de la práctica de ML; presumiblemente el apunte de Narias (que no tengo) lo dice también.

Lo que hicimos

Sweep en dos pasos:

- **Fase 1.3** ($k=1$, exploratoria): `{1e-4, 5e-4, 1e-3, 5e-3, 1e-2}`. Ganó **1e-3**.
- **Fase 2** ($k=5$, validación): mismos cinco valores + **extremos catastróficos** `{0.1, 10}` para mapear el rango de divergencia. Ganó **1e-3** otra vez.

Resultados Fase 2 (Ej2 README, sección “LR sweep”): - `lr=1e-3`: 96.22% (ganador). - `lr=1e-4`: 95.79% (converge pero lento, `best_ep=31`). - `lr=1e-2`: 94.71% (overshoot). - `lr=0.1`: 29% (diverge). - `lr=10`: 12.5% (NaN, accuracy random).

Sin mismatch — alineado con el espíritu del PDF

El PDF dice “ η muy grande diverge / η muy chico no avanza”; nuestro sweep mostró exactamente eso. Coincide.

5. Estrategia online / batch / mini-batch

Lo que dice la clase

No hay un slide explícito de batch vs online en los PDFs. El concepto de “online estricto = un update por muestra” es del apunte clásico de perceptrón simple (regla delta de Widrow-Hoff): $\Delta w = \eta(z - 0)x$.

Lo que hicimos

Ejercicio	Estrategia	Justificación
Ej1 (perceptrón simple)	Online estricto	Sigue la regla del apunte: un update por muestra. Ver <code>ejercicio1/.../nonlinear_perceptron.py</code> 238.

Ejercicio	Estrategia	Justificación
Ej2 / Ej3 (MLP)	Mini-batch con B por sweep	Vectorización de NumPy + estabilidad.

Sweep de batch size (Fase 1.4, k=1):

Batch	val_acc	best_ep	Tiempo	Comentario
16	96.74%	3	12.3 s	Ganador. Gradiente más ruidoso → converge antes.
32	96.58%	7	94.2 s	
64	96.74%	7	31.0 s	Empate con 16.
128	96.58%	11	21.4 s	

Sin mismatch — pero decisión “implícita”

No barrimos online vs full-batch en Ej2/3, solo mini-batch con distintos B. La razón: full-batch sobre 12k samples no aporta sobre mini-batch (mismo gradiente, más memoria); online estricto es lo que demostramos en Ej1.

6. Arquitectura: capas (M) y neuronas

Lo que dice la clase

regularizacion.pdf slide 8 (“Modelos de Machine Learning: capacidad”): - Capacidad = “habilidad o potencial de un modelo de aproximar una variedad determinada de funciones”. - Acciones que cambian capacidad: **elección de modelo, arquitectura del modelo, cantidad de características de input**. - Pregunta retórica: ¿qué tiene más capacidad, perceptrón simple no lineal o MLP? → MLP.

regularizacion.pdf slide 9 (curva U de generalización): - Underfitting / overfitting zone, **capacidad óptima** en el medio. - “Brecha de generalización” = gap entre error train y error eval.

Lo que hicimos

Sweep de arquitectura (Fase 1.1 + Fase 2):

Arch	val_acc	Comentario
[784, 100, 50, 10]	96.74% / 96.22%	Ganador por parsimonia. Empate técnico ($\Delta < 0.4\%$, std $\sim 0.4\%$).
[784, 128, 64, 10]	96.86% / —	
[784, 200, 100, 10] (wider)	— / 96.39%	+0.17 pp en val pero peor en test (-0.48 pp) y 3.6× más lento.
[784, 100, 50, 25, 10] (deeper)	— / 96.20%	Empate.
[784, 30, 10] (shallow)	— / 95.15%	Subajustado.

Decisión: [784, 100, 50, 10] por parsimonia (regla de Occam): cuando dos modelos rinden igual, gana el más simple/rápido.

Mismatch / sin backing directo

Capacidad óptima no fue cuantificable. El slide muestra la curva U pero no da una receta para elegir el punto óptimo. Nuestra elección por parsimonia es estándar en la práctica pero no aparece literal en el PDF.

El experimento de wider confirmó la idea de la clase: cuando aumentamos capacidad sin compensar con más datos/regularización, el modelo entró en overfitting (mejor en val, peor en test). Esto valida empíricamente la curva U del slide 9.

7. Epochs y criterio de convergencia (epsilon)

Lo que dice la clase

El PDF de regularización slide 14 muestra Early Stopping: la curva clásica de error de train bajando y error de validación primero baja y después sube; “parar cuando el error de validación deja de bajar”.

El criterio clásico del apunte para perceptrón simple (regla delta) es **epsilon absoluto sobre el error de train**: `if mse < epsilon: break`.

Lo que hicimos

Ejercicio	Criterio	Por qué
Ej1	Epsilon absoluto sobre MSE de train (<code>if mse < epsilon: break</code> en <code>nonlinear_perceptron.py:238</code>).	Sigue la regla del apunte para perceptrón simple.
Ej2 / Ej3	Early stopping sobre val_loss con <code>patience=10</code> (<code>mlp/network.py:200-210</code>).	Detecta overfitting (que <code>epsilon-train</code> no detecta).

epochs máximo: 50 en Ej2/3 (con `patience=10` corta antes; `best_epoch` promedio ~5).

Sin mismatch — alineado con el espíritu del PDF

Para Ej2/3 implementamos exactamente lo del slide 14 (early stopping). Para Ej1 usamos `epsilon-train` porque es lo que pide el apunte para perceptrón simple, y porque ahí no tenemos overfitting que detectar (modelo lineal de baja capacidad).

Tamaño de epsilon en Ej1: depende del config; típicamente del orden de $1e-3$ a $1e-4$ sobre MSE. No barrido sistemáticamente; elegido para que la corrida termine con error razonable pero sin bucle infinito.

8. Bias

Lo que dice la clase

Los PDFs no discuten cómo se incorpora el bias. Es decisión de implementación que presumiblemente cubre el apunte de Narias (que no tengo).

Lo que hicimos

Bias trick (`mlp/network.py:66-68`): a cada capa se le agrega una columna de 1's al input antes del producto matricial, y los pesos tienen shape $(n_{out}, n_{in} + 1)$. Es la convención estándar: el bias queda como un “peso más” y se actualiza con la misma regla de gradiente.

El bias NO se penaliza con L2 (`mlp/network.py:171-174`):

```
reg_grad[:, 1:] = l2 * W[:, 1:] # no penaliza bias
```

Sin mismatch — pero “no penalizar bias en L2” es decisión sin backing PDF

La práctica estándar en la literatura (Goodfellow et al. 2016, Chapter 7 — referenciado al final del PDF de regularización) es **no penalizar bias** porque el bias no contribuye a la varianza del modelo y penalizarlo sesga el ajuste. Lo que hicimos coincide con la práctica estándar pero no aparece literal en los PDFs.

9. Regularización / “Pack C” — matching con la clase

Esta sección cierra la última línea de ppt_notes.md: “Aca se hizo lo de ‘Pack C’ de estrategias. Son lo explicado en la clase de regularizacion. HACER UN MATCHING ACA DE TEORIA Y LO HECHO”.

Mapecto 1:1 — lo que la clase enseña vs. lo que implementamos

Técnica de la clase	Slide del PDF	Implementado en mlp/?	Usado en Ej3?
Early stopping	14	[OK] network.py:200–210 (val_loss + patience)	[OK] Activo en todos los configs (Ej2 y Ej3).
Data augmentation — gaussiano	18 (“Agregado de ruido (gaussiano)”)	[OK] network.py:160–165	[OK] Ganador: $\sigma=0.05$.
Data augmentation — rotaciones	18	[X] NO implementado	[X]
Data augmentation — traslaciones	18	[X] NO implementado	[X]
Data augmentation — cambios de escala	18	[X] NO implementado	[X]
L2 / Weight Decay	20-25 ($E_{reg} = E + (1/2)\lambda w ^2$)	[OK] network.py:168–174 (no penaliza bias)	[OK] Ganador con $\lambda=1e-4$.
Dropout	26 (mención: “Existen otros más”)	[OK] network.py:86–94 (inverted dropout)	Probado, no transfirió (gana val, pierde test).
Modelos de ensamble	26	[X] No aplicable al alcance del TP.	—
Aprendizaje semi-supervisado	26	[X] No aplicable.	—
Entrenamiento adversarial	26	[X] No aplicable.	—

Lo que la clase deja afuera (pero usamos)

- **lr_schedule (step decay)**: no es regularización en el sentido del PDF, pero sí está cerca del η adaptativo del PDF de optimizadores. Lo agrupamos en “Pack C” por implementación.

Resultado del experimento (resumido del Ej3 README)

Config	Δ test
base_extra_data (con more_digits.csv)	+10.06 pp ← dominante
+ L2 ($1e-4$)	+0.20 pp
+ Dropout ($p=0.2$)	-0.08 pp (no transfiere)
+ L2 + augmentation gaussiana ($\sigma=0.05$)	+0.52 pp ← ganador final, 96.88%

Lección clave que el doc 1 ya deja implícita pero conviene ver: la fórmula L2 del PDF está implementada literal (incluso $(1/2)\lambda ||w||^2$ con la constante 1/2 absorbida en la derivada $\partial E_{\text{reg}}/\partial w = \partial E/\partial w + \lambda w$); el augmentation gaussiano del slide 18 es exactamente lo que implementamos; lo que **faltó** son las otras tres formas de augmentation (rotaciones, traslaciones, escalas) que el slide 18 lista pero nosotros no implementamos. **Ese es el principal “no llegamos al 98%” — coincide con la última hipótesis del Ej3 README.**

Pregunta de ppt_notes.md — “¿qué decisiones se tomaron?”

Pack C se activó **en el orden recomendado por la clase** (early stopping siempre activo; después L2; después augmentation; dropout probado pero descartado). El criterio para elegir el ganador fue el **test accuracy** (digits_test.csv), no el val K-fold, justamente porque la lección de Ej2 fue que val K-fold subestima el shift train→test (ver bitácora, sección “Final eval Ej 2”).

Resumen ultra-corto (cheatsheet mental)

Decisión	Backing en PDFs cátedra	Mismatch a marcar
ReLU + softmax (Ej2/3)	Indirecto (slide 2 lista activaciones, sin profundizar)	Elección estándar de literatura, no rastreable a slide concreto.
He / Xavier / Uniform	HTML de weight init que vino con el TP	Nuestro Xavier usa $\sqrt{1/fan_in}$, no $\sqrt{2/(n_in+n_out)}$ del HTML.
Adam (default $\alpha=1e-3$, $\beta=(.9,.999)$, $\epsilon=1e-8$)	PDF optimizadores p.13 — coincide exacto	—
LR=1e-3 + sweep con extremos	Indirecto (η adaptativo motiva el sweep)	—
Mini-batch B=16 (Ej2/3), online (Ej1)	Mini-batch no está en PDFs; online es regla del apunte	—
Arch [784, 100, 50, 10] por parsimonia	Slides 8-9 sobre capacidad / curva U	“Parsimonia” no es regla literal del PDF.
Early stopping en Ej2/3 / epsilon en Ej1	Slide 14 (early stopping) + apunte (epsilon)	—
Bias trick + no penalizar bias en L2	No está en PDFs	Práctica estándar (Goodfellow Cap. 7), no en slides.
L2 con $\lambda=1e-4$	PDF regularización slides 20-25 — fórmula exacta	—
Augmentation gaussiana ($\sigma=0.05$)	Slide 18 — gaussiano explícito	Slide 18 también lista rotaciones/traslaciones/escalas; NO las implementamos.
Dropout ($p=0.2$) — descartado	Slide 26 — solo mencionado	Probamos pero no transfirió a test.